Sindy's [EECS492] Mid

Agent and Environment

PEAS: Performance, Environment, Actuator, Sensor
Autonomy, Rationality, Coherence

Environment Properties

* Fully vs. Partially Observable
* Deterministic vs. Stochastic
Deterministic if the environment is completely determined by the current state and the action executed by the agent
* Episodic vs. Sequential
* Static vs. Dynamic
* Discrete vs. Continuous
* Single-Agent vs. Multi-Agent
* Known vs Unknown
Knowledge of rules of the environment
* Episodic vs. Sequential

Agent Design

* Table Look-up Agent
* Simple Reflex Agent
Selects actions on the basis of the current percepts.
* Model-based Reflex Agent
Depends on the percepts and thereby reflects some of the unobserved aspects of the current state.
* Goal-based Agent
* Utility-based Agent
Utility function being more complex than accomplishing goal.
Learning Agent

Search

Well-defined Problems:
* Initial state
* A set of available actions defined as successor function
* Goal test
* Path cost function
* Problem solving as a search process paradigm (initial sate, goal state, operators, path cost)

Branching factor (b): # of successors each node has
Depth (d): depth of the shallowest solution

Search Performance Criteria

Completeness: guaranteed to find a solution if it exists
Optimality: finds the minimum path
Time complexity
Space complexity

Uniformed Search Strategies

* Breadth-first
* Uniform-cost
Expand the node with minimum path cost
* Depth-first
* Depth-limited
Cut off search at a predetermined depth
* Iterative-Deepening * Bidirectional * Graph Search
Skip examined node

Informed Search Strategies

* Priority-first (greedy)
* A*
path length so far + estimate path length
* Iterative-deepening A* * Stochastic
Random-restart hill climbing
Simulated Annealing: Hill climbing, but take worse-appearing steps with some probability. Decrease tempreture gradually
Genetic Algorithm: Parallel hill climbing

Constraint Satisfaction Search

Consistent or legal: an assignment that does not violate any constraint
Complete: an assignment in which every variable is mentioned
Solution: a complete and consistent assignment
* Minimum Remaining Values heuristic:
Choose variable with fewest legal values remaining (most-constrained variable) * Forward Checking:
Whenever X is assigned, examine unassigned Y connected to X by a constraint and delete form Y's domain any value inconsistent with X
* Constraint Propagation
Propagating the implications of a constraint on one variable onto other variables.
* Node/Arc/Path consistency,
* Backjumping:
Keep track of variables constraint-checked with current variable

Logic

* Logic: Knowledge Representation Language (KRL) with well-defined syntax & semantics
* Knowledge Base (KB): a set of sentences using KRL
* Knowledge-based agents (goal-based):
  1. tell KB what it perceives
  2. ask KB the action to perform
  3. tell KB which action is chosen
* Interface: TELL / ASK
* Syntax: defines the legal sentences
* Semantics: specify truth or falsity for every sentence in a world

Entailment and Inference

Entailment: $KB \models \alpha$ (KB entails α)
Inference: $KB \vdash \alpha$ (KB derives α)
Sound: every inference can be entailed
Complete: every entailment can be derived

Validity and Satisfiability

Valid: always true
Satisfiable: true in some model

Propositional Logic (PL) (0th order)

* Literal: an atomic sentence or negation
Syntax:
Sentence: Atomic, Complex
AtomicSentence: T, F, Symbol (P, Q, R)
ComplexSentences: constructed with sentences and operators ($\neg$, $\wedge$, $\vee$, $\Rightarrow$, $\Leftrightarrow$)
Semantics:
Define truth of any sentence wrt a model recursively. Compositional semantics.
Entailment method: Model checking (truth table method) propositioanal theorem proving: a proof is a sequence of applications of sound inference rules (more efficient)

Inference Rules $\frac{premise}{conclusion}$

Modules Ponens:

$$\frac{\alpha \Rightarrow \beta, \alpha}{\beta}$$

Extended Modules Ponens:

$$\frac{\alpha_1 \wedge \cdots \wedge \alpha_m \Rightarrow \beta, \alpha_1, \cdots, \alpha_m}{\beta}$$

Generalized Modules Ponens:

$$\frac{\alpha_1 \wedge \cdots \wedge \alpha_m \Rightarrow \beta, \alpha_i}{\alpha_1 \wedge \cdots \wedge \alpha_{i-1} \wedge \alpha_{i+1} \wedge \cdots \wedge \alpha_m \Rightarrow \beta}$$

And-elimination:

$$\frac{\alpha \wedge \beta}{\alpha}$$

Biconditional-elim:

$$\frac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$$

$$\frac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$$

Monotonicity:

$$KB \models \alpha \text{ then } KB \wedge \beta \models \alpha$$

Resolution and Refutation

Unit resolution

$$\frac{l_1 \vee \cdots \vee l_k, \neg l_i}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \cdots \vee l_k}$$

Full resolution

$$\frac{l_1 \vee \cdots \vee l_k,\ m_1 \vee \cdots \vee m_n}{l_1 \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_n}$$

Refutation: to show $KB \models \alpha$ by showing that $KB \wedge \neg\alpha$ leads to contradiction

Proof as Search

State: Set of sentences
Initial state: KB
Successors: all possible applications of sound inference rules

Horn clauses and definite clauses

Clauses: a disjunction of literals
Definite clauses: exactly one is positive.
Horn clauses: at most one is positive
Goal clauses: none is positive

Forward Chaining (naive)
1. Loop through implicate sentences $P_1 \wedge P_2 \wedge \cdot \wedge P_n \Rightarrow Q$. If all premises of an implication are true, the conclusion can be added to the known fact.
2. If any new conclusions added, go to 1.
3. When no more conclusions, see if query is answered in the resulting sentences.
Note: resolving two Horn clauses gives a Horn clause
Forward-chaining determines if a single proposition symbol q is entailed by a KB of definite clauses. Sound and Complete. Runs in a linear time as the size of KB. A form of data-driven reasoning

Backward-chaining is a form of goal-directed reasoning.

Reasoning

Brute-force (truth-table modelchecking)
Inference: forwardchaining, backward chaining, resolution/refutation

First-Order Logic (FOL)

First order logic consists of a set of objects and their interpretation; allows quantification over logic.
Syntax: Sentence: Atomic, Complex, Quantifier + Sentence
Atomic: Predicate(Term), Term = Term
Term: (a logical expression that refers to an object) Function(Term), Constant, Variable
Quantifier: universal $\forall$, existential $\exists$
Constant(higher case): objects
Variable(lower case)
Predicate: relation, returns a booleen
Functions: returns a variable

PL vs. FOL
* In PL, model assigns truth values to propositional symbols
* In FOL, model provide interpretation of functions and relations

Unification

A form of pattern matching
e.g. Knows(Cousin(y), Mittens) and Knows(Friends(Spot, x)) Fails.

Conjunctive Normal Form (CNF): AND of ORs

1. Eliminate $\Leftrightarrow$ by replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
2. Eliminate $\Rightarrow$ by replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$
3. Move $\neg$ inward using double-negation elimination and De Morgan rule
4. Standardize variables: to ensure every variable has its unique name
5. Eliminate existential via Skolemization: use Skolem function of all universal quantifiers to replace existential quantifiers
6. Drop universal quantifiers
7. Distribute $\vee$ over $\wedge$

Prolog

* Use backward chaining
* Relations and constants are lowercase, and variables are capitalized!
* ":-" is <=; ',' is AND; '.' indicates the end of a sentence.
* ";" is OR (seldom needed)

EECS492 Final

Planning

~agent: goal-based
knowledge-based
~task: { init/goal state
action

⇒ find a course of action

Classical Planning

fully observable, finite deterministic, static, discrete, single~, known

*state: conjunctions of function free ground positive literals with closed world semantics. (no ¬)

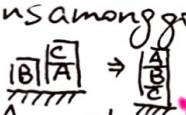
*goal: conjunctions of literals or negations

Planning Domain Def Lan

action { name
precondition
effect

State space search

{ progression (heuristic)
regression (only consider relevant action; deal with interactions among goal states)



Sussman Anomaly

Graph Plan

mutex Action:
Inconsistent effects
(produce complementary propositions)
Interference (one negates the precondition of other)
Competing Needs (complementry preconditions)

Proposition: Compliments
Inconsistent Support

All pairs of actions that produce the pair are mutex

*nogood: impossible sub-goal combinations.

Propositional Planning: SAT

int state ∧ goal* ∧ domain axiom

successor state ~, pre-condition ~, action exclusion

Partial Order Planning

causal link

solution { Complete } exist
consistent: no contradiction/constraint

threat (衝突) { promotion
demotion

Planning Under Uncertainty

sensorless/conformant ~

conditional/contingent ~
*percept

replanning
{ action monitoring
plan monitoring

Probability

Conditional ~ $Pr(A|B) = \frac{Pr(A,B)}{Pr(B)}$

Bayes' Theorem

$P(A|B) = P(A,B)/P(B)$

$= P(B|A)P(A)/P(B)$ $\sum_a P(B|A=a)$

Bayesian Networks

Independence: $P(A|B) = P(A)$

$\Rightarrow P(A,B) = P(A)P(B)$

Conditional ~: $P(A|B,C) = P(A|C)$

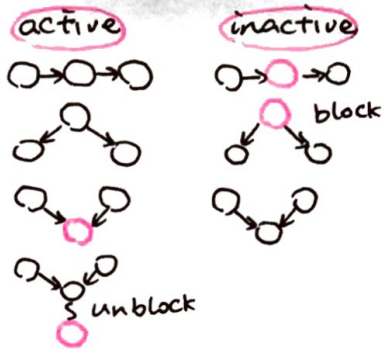
Applying chain rule:

$P(x_1,\dots,x_n) = P(x_1|x_2,\dots,x_n)\cdot P(x_2|x_3,\dots,x_n)\cdot P(x_3|x_4,\dots,x_n)\cdots$

d-seprated ⇒ conditional independence

$IS(a,S,b)$ iff no active path between each tuple is active

iff $\forall$ undirected path blocked (all) no path unblocked.



Law of Total Pr

$P(W) = \sum_r \sum_s \sum_c P(W,r,s,c)$

(can move summation inward)

Known Value Iteration

$U(i) = R(i) + \gamma \max_a \sum_k T_{ik}(a) U(k)$

transition probability given choice action

Bellman Equation

Variable Clustering

Bayesian Network

⇒ polytree

Markov Decision Process

$U'(s) = R(s) + \gamma \max_a \sum P(s'|s,a) U(s')$

Value of Perfect Info

$\sum_e Pr(E'=e|E) EU(\alpha_e|E,e) - EU(\alpha,E)$

Uncertainty/Entropy

$H(X) = -\sum P(x_i)\log_2 P(x_i)$

$Gain(S|A) = H(S) - H(S|A)$

Learning

Decision Tree

Entropy $I(q) = -q\log_2 q - (1-q)\log_2(1-q)$

Before: $I(p/(p+n))$

After: $\frac{p_t+n_t}{p+n} I\left(\frac{p_t}{p_t+n_t}\right) + \frac{p_f+n_f}{p+n} I\left(\frac{p_f}{p_f+n_f}\right)$

Ensemble Learning

AdaBoost: great for weak learning algorithm; (large number of iteration

For each iteration k,

Find $H_k(x)$ with $W_k(x)$

$\varepsilon_k = \sum W(\text{incorrect})$

$\alpha_k = \ln(1-\varepsilon_k/\varepsilon_k)$

update correct training instance

$W_{k+1} = W_k \times \varepsilon_k/(1-\varepsilon_k)$

*re-normalize

$H_{FINAL}(x) = \text{sign}(\sum \alpha_i H_i(x))$

Reinforcement Learning

(Q-Learning) (Adaptive Dynamic Programming learns model)

*model free

*no reward/transition func

$Q = q + \alpha\cdot(R + (\gamma\, q') - q)$

for state and action — best q'

{ exploration
exploitation

Probably Approximately Correct Learning

n attributes (input) that take k value each.

output one of n class es

⇒ m^{k^n} possible hypothesis

Given x examples, still have m^{k^n}/m^x hypothesis that are consistent.

Approximately correct.

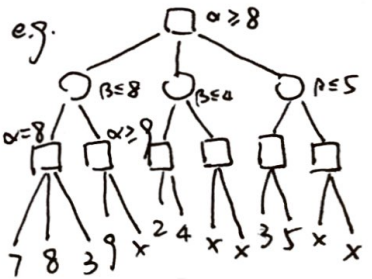
Naive Bayes

greatest posterior probability (joint probability of the example given that class, and of observing each attribute value)

$\hat{y} = \text{argmax}\, p(C_k)\prod p(x_i|C_k)$

Alpha-Beta Pruning

adversarial MINIMAX

$O(b^m) \Rightarrow O(b^{m/2}) \sim O(b^{3m/4})$

e.g.